

PYTHON UYGULAMA GELİŞTİRME

Mimariler, Proje Dizini, Paketler, Teknolojiler ve Uçtan Uca Başlangıç Rehberi

Python ile fikirden çalışan, test edilen, paketlenen ve dağıtılan uygulamaya geçiş için geliştirici başvuru dokümanı

Hedef kitle	Başlangıç ve orta düzey Python geliştiricileri; proje liderleri; eğitimciler
Kapsam	Mimari, klasör yapısı, paket seçimi, test, güvenlik, dağıtım, işletim sistemi farkları
Yaklaşım	Basitten karmaşığa; gereksinim ve risk odaklı teknoloji seçimi
Kaynak temeli	Python ve ilgili teknolojilerin resmî dokümantasyonları; mimari desenlerin özgün kaynakları

Not: “Eksiksiz mimari listesi” pratikte mümkün değildir; yazılım mimarileri çok geniş ve sürekli gelişen bir alandır. Bu belge Python uygulamalarında en sık kullanılan ve karar vermeyi doğrudan etkileyen mimarileri kapsamlı biçimde ele alır.

İçindekiler

1. 1. Belgenin kullanım biçimi
2. 2. Yazılım fikrinden teknik plana geçiş
3. 3. Uygulama türünü belirleme
4. 4. Mimari seçme ilkeleri
5. 5. Python’da kullanılan başlıca mimariler
6. 6. Standart proje ve dosya dizini
7. 7. Uygulama türlerine göre örnek klasör yapıları
8. 8. Paket ve teknoloji seçimi
9. 9. İşletim sistemi farklılıkları
10. 10. Geliştirme ortamı ve ilk kurulum
11. 11. Veri katmanı ve veritabanı seçimi
12. 12. Test stratejisi
13. 13. Güvenlik, yapılandırma ve gizli bilgiler
14. 14. Loglama, hata yönetimi ve gözlemlenebilirlik
15. 15. Paketleme ve dağıtım
16. 16. CI/CD ve sürüm kontrolü
17. 17. Mimariye göre örnek uygulamalar
18. 18. Proje başlangıç şablonları
19. 19. Karar kontrol listeleri
20. 20. Resmî ve temel kaynaklar

1. Belgenin Kullanım Biçimi

Bu rehber bir teknoloji kataloğu değildir. Amaç, geliştiricinin “hangi paketi kurmalıyım?” sorusundan önce “hangi problemi, hangi kullanıcı için, hangi kalite ölçütleriyle çözüyorum?” sorusunu yanıtlamasını sağlamaktır. Mimari ve paketler, iş ihtiyacına hizmet eden araçlardır.

- Küçük bir otomasyon betiği için Clean Architecture kurmak gereksiz karmaşıklık yaratabilir.
- Uzun süre yaşayacak, birden fazla geliştiricinin çalışacağı ve farklı veri kaynaklarına bağlanacak bir sistemde tek dosyalı yaklaşım teknik borcu hızla artırır.
- Framework seçimi mimari seçimi değildir. FastAPI, Django, Flask veya PySide6 birer teknoloji/framework seçimidir; bunların içinde katmanlı, hexagonal veya özellik-temelli yapı uygulanabilir.
- Her dış bağımlılık güncelleme, güvenlik ve uyumluluk maliyeti taşır. Standart kütüphane yeterliyse ek paket zorunlu değildir.

2. Yazılım Fikrinden Teknik Plana Geçiş

Adım	Sorulacak temel soru / çıktı
Problem tanımı	Mevcut durum nedir? Sorun kimde, ne sıklıkta ve hangi kayba yol açıyor?
Hedef kullanıcı	Kullanıcı rolleri, teknik yeterlilikleri, cihazları ve erişilebilirlik ihtiyaçları belirlenir.
Başarı ölçütü	Başarı; hız, doğruluk, kullanım oranı, hata azalması, maliyet veya zaman tasarrufu gibi ölçülebilir göstergelerle tanımlanır.
Kapsam ve kapsam dışı	İlk sürümün yapacağı ve özellikle yapmayacağı işler açıkça yazılır.
MVP	Tek bir temel kullanıcı yolculuğunu uçtan uca çalıştıran en küçük değerli sürüm seçilir.
Fonksiyonel gereksinimler	Kullanıcıların gerçekleştireceği işlemler ve sistem davranışları tanımlanır.
Fonksiyonel olmayan gereksinimler	Performans, güvenlik, yedekleme, erişilebilirlik, çevrimdışı çalışma, bakım ve mevzuat koşulları belirlenir.
Veri modeli	Temel varlıklar, ilişkiler, veri sahipliği, saklama süresi ve gizlilik sınıfları çıkarılır.
Mimari ve teknoloji	Değişim ihtimali yüksek noktalar ile sabit iş kuralları ayrıştırılır; uygun mimari ve teknoloji seçilir.
Risk ve prototip	En belirsiz veya pahalı teknik varsayım küçük bir prototiple doğrulanır.
Teslim planı	İşler özellik, kullanıcı hikâyesi veya dikey dilimler hâlinde sıralanır.
İşletim planı	Loglama, izleme, güncelleme, yedekleme, veri taşıma ve geri alma süreçleri daha başta düşünülür.

2.1 Örnek fikir çözümleme: Okul içi kitap paylaşım sistemi

- Problem: Öğrencilerin evde kullanılmayan kitapları okul içinde güvenli biçimde paylaşamaması.
- Hedef kullanıcı: Öğrenci, öğretmen/moderatör, sistem yöneticisi.
- MVP: Kayıt, kitap ekleme, kitap arama, talep oluşturma, teslim durumunu güncelleme.
- Başarı ölçütleri: Paylaşılan kitap sayısı, tamamlanan teslim oranı, geciken iade oranı, aktif kullanıcı oranı.

- Kritik kurallar: Yalnız okul öğrencileri; kişisel verilerin asgari tutulması; işlem kayıtlarının denetlenebilir olması.
- Teknik başlangıç: Tek okul ve orta trafik için modüler monolit + Django veya FastAPI; daha sonra ihtiyaç olursa servis sınırları ayrılabilir.

3. Uygulama Türünü Belirleme

Uygulama türü	Örnek	Tipik araçlar	Başlangıç yapısı
Betik / otomasyon	Dosya dönüştürme, yedekleme, rapor üretme	argparse, pathlib, csv, shutil, logging	Tek dosya → modüler paket
CLI uygulaması	Geliştirici aracı, veri içe aktarma, yönetim komutu	argparse veya Typer/Click, rich	Komutlar + servisler
Web API	Mobil/ön yüz uygulamalarına veri ve işlem servisi	FastAPI/Flask/Django Ninja, Pydantic, SQLAlchemy	Katmanlı / Clean / Hexagonal
Sunucu taraflı web	Panel, içerik sistemi, e-ticaret, okul sistemi	Django, templates, ORM	MVT + özellik-temelli modüller
Masaüstü	Çevrimdışı yönetim, raporlama, cihaz entegrasyonu	PySide6, Tkinter, sqlite3/SQLAlchemy	MVC/MVVM veya katmanlı
Veri analizi / ETL	Temizleme, dönüştürme, raporlama	pandas, polars, openpyxl, pyarrow	Pipeline / modüler fonksiyonlar
Arka plan işçisi	E-posta, rapor, dosya işleme, zamanlanmış görev	Celery/RQ/ARQ, Redis	Kuyruk + worker
Makine öğrenmesi	Model eğitim, değerlendirme, çıkarım	scikit-learn, PyTorch, MLflow	Pipeline + servis katmanı
Kütüphane / SDK	Başka projelerin kullanacağı tekrar kullanılabilir kod	pyproject.toml, build, twine	Paket odaklı API tasarımı
Dağıtık sistem	Bağımsız ölçeklenen iş alanları	Mesajlaşma, API gateway, servisler	Mikroservis / olay güdümlü

4. Mimari Seçme İlkeleri

- Mimari; dosyaları estetik biçimde ayırmak değil, değişimin etkisini sınırlamak ve bağımlılıkları yönetmektir.
- İş kuralları, kullanıcı arayüzü ve veritabanı aynı hızda değişmez. En kalıcı kuralların en az bağımlılığa sahip olması hedeflenir.
- Ölçek yalnız trafik değildir. Ekip büyüklüğü, özellik sayısı, veri hacmi, dağıtım sıklığı ve entegrasyon sayısı da ölçeğin parçasıdır.
- Erken mikroservis çoğu projede operasyonel maliyeti artırır. Modüler monolit genellikle daha güvenli başlangıçtır.
- Mimari kararlar geri döndürülebilirlik açısından değerlendirilmelidir. Veritabanı ve framework gibi zor değişen kararlar kanıtlarla alınmalıdır.

Soru	Basit yapı lehine	Daha disiplinli mimari lehine
Proje ömrü	Kısa süreli	Yıllarca bakım
Ekip	Tek geliştirici	Birden fazla ekip
İş kuralı	Az ve basit	Yoğun ve değişken
Dış entegrasyon	Yok/az	Birden fazla API, DB, cihaz
Test ihtiyacı	Düşük risk	Kritik işlem ve yüksek risk
Dağıtım	Tek cihaz	Sunucu, bulut, çoklu ortam
Uyumluluk	Tek platform	Windows/Linux/macOS

Soru	Basit yapı lehine	Daha disiplinli mimari lehine
Denetlenebilirlik	Sınırlı	Kayıt ve iz zorunlu

5. Python'da Kullanılan Başlıca Yazılım Mimarileri

Mimari / yaklaşım	Ne zaman?	Güçlü yön	Risk	Örnek
Tek dosyalı betik	Küçük, tek amaçlı ve kısa ömürlü işlemler.	Hızlı başlangıç.	Büyüdüğünde test ve bakım zorlaşır.	Dosya yeniden adlandırma aracı.
Modüler yapı	Kodun fonksiyon ve modüllere ayrıldığı küçük/orta projeler.	Basitlik ile düzen arasında iyi denge.	Sınırlar bilinçsiz kurulursa döngüsel import oluşur.	Rapor üretici, ETL aracı.
Paket tabanlı yapı	Tekrar kullanılabilir kütüphane veya çok modüllü uygulama.	Açık genel API ve dağıtılabilirlik.	Paketleme ve sürüm uyumluluğu gerekir.	Kurumsal yardımcı kütüphane.
Katmanlı mimari	Sunum, uygulama, alan ve altyapı sorumluluklarının ayrılması.	Bakım, test ve teknoloji değişimi kolaylaşır.	Aşırı katman, basit projede bürokrasi yaratır.	Okul yönetim masaüstü uygulaması.
MVC	Model, View ve Controller ayrımı; özellikle arayüz uygulamaları.	Arayüz ile veri/iş mantığını ayırır.	Framework yorumları farklı olabilir.	Flask tabanlı panel, masaüstü arayüz.
MVT/MTV	Django'nun model-template-view adlandırması.	Django ekosistemiyle hızlı geliştirme.	View terimi klasik MVC controller rolüne yaklaşır.	Yönetim paneli, portal.
MVVM	View, ViewModel ve Model; veri bağlama odaklı UI.	UI test edilebilirliği ve durum yönetimi.	Küçük arayüzlerde gereksiz karmaşıklık.	PySide6/QML masaüstü uygulaması.
Clean Architecture	İş kurallarını framework ve altyapıdan bağımsız tutar.	Test edilebilirlik ve teknoloji bağımsızlığı.	Daha fazla soyutlama ve dosya.	Uzun ömürlü iş sistemi.
Hexagonal / Ports & Adapters	Uygulama çekirdeğinin portlarla tanımlanıp adaptörlerle dış dünyaya bağlanması.	DB/API/UI değiştirilebilirliği ve izolasyon.	Port sayısı gereksiz artırılabilir.	Birden çok veri kaynağına bağlı sistem.
Onion Architecture	Domain merkezli halkalar ve içe doğru bağımlılık.	İş alanını korur.	Clean/Hexagonal ile kavramsal örtüşme kafa karıştırabilir.	Domain yoğun kurumsal uygulama.
Domain-Driven Design yaklaşımı	Karmaşık iş alanını bounded context, aggregate, value object gibi kavramlarla modelleme.	İş dili ile kodun uyumunu güçlendirir.	Basit CRUD projelerinde aşırı maliyetlidir.	Bütçe, stok, sınav veya proje yönetimi.
Modüler monolit	Tek dağıtım birimi içinde güçlü modül sınırları.	Mikroservis maliyeti olmadan ayrışma.	Sınırlar uygulanmazsa sıradan monolite dönüşür.	Kurumsal yönetim sistemi.
Mikroservis	İş alanlarının bağımsız servis ve dağıtım birimlerine ayrılması.	Bağımsız ölçekleme ve dağıtım.	Ağ, gözlemlleme, veri tutarlılığı ve DevOps maliyeti.	Çok ekipli yüksek ölçekli platform.
Olay güdümlü mimari	Bileşenlerin olay yayımlayıp tüketmesi.	Gevşek bağlılık ve asenkron işleme.	Sıralama, tekrar, izlenebilirlik ve eventual consistency.	Bildirim, entegrasyon, işlem akışı.

Mimari / yaklaşım	Ne zaman?	Güçlü yön	Risk	Örnek
Pipeline / Pipes and Filters	Verinin ardışık bağımsız adımlardan geçmesi.	Adımların yeniden kullanımı ve test edilmesi.	Hata yönetimi ve ara veri formatı gerekir.	ETL, medya dönüştürme.
Plugin mimarisi	Çekirdek uygulamanın eklentilerle genişletilmesi.	Yeni özelliklerin çekirdekten ayrılması.	Sürüm ve güven sınırı yönetimi gerekir.	IDE eklentisi, rapor modülleri.
Serverless / Function as a Service	Olay veya HTTP isteğiyle kısa çalışan fonksiyonlar.	Operasyon azaltma ve isteğe bağlı ölçekleme.	Soğuk başlangıç, sağlayıcı bağımlılığı, çalışma sınırları.	Dosya yüklendiğinde işleme.
CQRS (gerektiğinde)	Okuma ve yazma modellerini ayırma.	Karmaşık okuma yüklerinde esneklik.	Tutarlılık ve operasyon karmaşıklığı.	Yoğun raporlama ve işlem sistemi.

5.1 Katmanlı mimarinin önerilen bağımlılık yönü

```

presentation/ -> application/ -> domain/
                        ^
                        |
infrastructure/ -----+

```

Kural: domain katmanı FastAPI, Django, SQLAlchemy, PySide6 gibi dış teknolojileri bilmemelidir.

Clean Architecture'ın temel "Dependency Rule" yaklaşımında kaynak kod bağımlılıkları içe, daha yüksek seviyeli politikalara doğru yönelir. Hexagonal Architecture ise çekirdeği portlar üzerinden UI, veritabanı ve dış servis adaptörlerine bağlar. Bu iki yaklaşım aynı değildir; ancak ikisi de iş kurallarını teknik ayrıntılardan korumayı amaçlar. [12][13]

6. Standart Python Proje ve Dosya Dizini

```

project-name/
├── README.md
├── LICENSE
├── pyproject.toml
├── .python-version          # isteğe bağlı
├── .gitignore
├── .env.example
├── requirements.txt         # araç tercihinə göre
├── requirements-dev.txt    # araç tercihinə göre
├── src/
│   └── project_name/
│       ├── __init__.py
│       ├── __main__.py    # python -m project_name
│       ├── config.py
│       ├── domain/
│       ├── application/
│       ├── infrastructure/
│       ├── presentation/
│       └── shared/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── docs/
├── scripts/
├── migrations/             # veritabanı varsa
├── Dockerfile              # gerekiyorsa
├── compose.yaml            # gerekiyorsa
└── .github/workflows/      # GitHub Actions kullanılıyorsa

```

Dosya / dizin	Görevi
README.md	Amaç, kurulum, kullanım, yapılandırma, test ve dağıtım talimatları.
LICENSE	Kullanım ve dağıtım hakları. Özel projede kurum

Dosya / dizin	Görevi
	politikasıyla belirlenir.
pyproject.toml	Modern paketleme metadatası, build sistemi ve ruff/pytest/mypy gibi araçların ortak yapılandırması.
.gitignore	Sanal ortam, önbellek, gizli dosyalar ve üretilen çıktıların sürüm kontrolüne girmemesini sağlar.
.env.example	Gerçek sır içermeyen ortam değişkeni şablonu.
src/	Uygulama kaynak kodu. src-layout, yanlışlıkla proje kökünden import riskini azaltır.
__init__.py	Klasörü düzenli Python paketi hâline getirir; paket API'sini kontrollü sunabilir.
__main__.py	Paketi <code>python -m paket_adi</code> biçiminde çalıştırır.
tests/	Üretim kodundan ayrı otomatik testler.
docs/	Mimari karar kayıtları, API ve kullanıcı dokümantasyonu.
scripts/	Tek seferlik bakım, veri taşıma ve geliştirici görevleri.
migrations/	Veritabanı şema değişiklikleri; ör. Alembic veya Django migrations.
Dockerfile / compose.yaml	Konteyner imajı ve yerel çoklu servis ortamı.
CI dosyaları	Test, kalite ve paketleme işlemlerini otomatikleştirir.

Python Packaging User Guide, paketlenecek projeler için pyproject.toml tabanlı yapı ve gerekli proje dosyalarını resmî olarak açıklar. pyproject.toml yalnız paketleme için değil, linter ve tip denetleyici gibi araçların yapılandırılması için de kullanılır. [2][3]

7. Uygulama Türlerine Göre Örnek Klasör Yapıları

7.1 CLI / otomasyon

```

file_reporter/
├── pyproject.toml
├── src/file_reporter/
│   ├── __init__.py
│   ├── __main__.py
│   ├── cli.py
│   ├── commands/
│   ├── services/
│   └── adapters/
└── tests/

```

- Komut satırı argümanlarını cli.py alır.
- Asıl iş kuralları services/ içinde tutulur.
- Dosya sistemi veya API erişimi adapters/ içinde ayrıştırılır.
- Tek komutlu küçük araçlarda argparse yeterlidir; zengin komut grupları için Click/Typer değerlendirilebilir.

7.2 FastAPI tabanlı Web API

```

school_api/
├── pyproject.toml
├── src/school_api/
│   ├── main.py
│   ├── api/
│   │   ├── dependencies.py
│   │   └── routers/
│   ├── schemas/
│   ├── application/
│   │   ├── use_cases/
│   │   └── ports/
│   ├── domain/
│   ├── infrastructure/
│   │   ├── db/
│   │   └── repositories/
│   └── config.py
├── tests/
└── migrations/

```

FastAPI'nin resmî “Bigger Applications – Multiple Files” rehberi, büyük uygulamalarda APIRouter ve paketlere ayrılmış çok dosyalı yapı kullanılmasını gösterir. Ayarlar ayrı config.py modülünde ve pydantic-settings ile yönetilebilir. [4][5]

7.3 Django uygulaması

```

portal/
├── manage.py
├── pyproject.toml
├── config/
│   ├── settings/
│   │   ├── base.py
│   │   ├── development.py
│   │   └── production.py
│   ├── urls.py
│   └── wsgi.py / asgi.py
├── apps/
│   ├── accounts/
│   ├── books/
│   └── loans/
├── templates/
├── static/
└── tests/

```

- Her iş alanı ayrı Django app olarak düzenlenebilir.
- Model, template ve view sınırları Django'nun MTV/MVT yaklaşımına göre kurulabilir.
- Ayarlar geliştirme ve üretim ortamlarına göre ayrılmalıdır.
- Django'nun resmî açıklamasında framework MTV olarak adlandırılır; URL yönlendirme ve framework mekanizması controller rolüne yaklaşır. [6]

7.4 PySide6 masaüstü uygulaması

```

desktop_manager/
├── pyproject.toml
├── src/desktop_manager/
│   ├── main.py
│   ├── presentation/
│   │   ├── windows/
│   │   ├── widgets/
│   │   └── viewmodels/
│   ├── application/
│   ├── domain/
│   ├── infrastructure/
│   └── resources/
├── tests/
└── packaging/

```

Qt for Python, Qt'nin resmî Python bağlarını sunar; PySide6 Qt 6 API'lerini Python'dan kullanmayı sağlar. Resmî başlangıç rehberi sanal ortam kullanımını önerir. [14]

7.5 Veri işleme / ETL

```

student_etl/
├── pyproject.toml
├── src/student_etl/
│   ├── cli.py
│   ├── pipelines/
│   ├── extractors/
│   ├── transformers/
│   ├── loaders/
│   ├── validators/
│   └── config.py
├── tests/fixtures/
├── data/raw/      # genellikle Git dışında
├── data/processed/ # genellikle Git dışında
└── reports/

```

8. Paket ve Teknoloji Seçimi

İhtiyaç	Standart kütüphane / temel seçenek	Yaygın üçüncü taraf seçenekler	Seçim notu
Sanal ortam	venv	virtualenv, conda	Genel uygulamalarda venv çoğu zaman yeterlidir.
Paket kurma	pip	uv, Poetry, PDM, Hatch	Takımın kilit dosyası ve yayın sürecine göre seçin.
Proje yapılandırması	pyproject.toml	Araçların kendi bölümleri	Tek merkezde yapılandırma tercih edilir.
CLI	argparse	Click, Typer, Rich	Küçük araçta argparse; komut gruplarında Typer/Click.
Web API	http.server (sınırlı)	FastAPI, Flask, Django Ninja	Şema ve async ihtiyacına göre.
Tam web	—	Django, Flask	Django bütünleşik; Flask daha minimal.
Masaüstü	tkinter	PySide6, wxPython, Kivy	Native görünüm, lisans ve dağıtım dikkate alınır.
HTTP istemci	urllib	requests, httpx	Async istek gerekiyorsa httpx.
Veri doğrulama	dataclasses, typing	Pydantic, attrs	Dış veri sınırlarında doğrulama önemlidir.
ORM	sqlite3	SQLAlchemy, Django ORM	Teknoloji bağımsız veri katmanı için SQLAlchemy.
Migration	—	Alembic, Django migrations	Şema değişiklikleri kodla izlenmelidir.
Test	unittest	pytest, hypothesis, coverage	pytest eklenti ekosistemi güçlüdür.
Kod kalitesi	compileall	Ruff, Black, mypy, pre-commit	Az araçla tutarlı otomasyon.
Veri	csv, json	pandas, polars, openpyxl, pyarrow	Dosya boyutu ve bellek ihtiyacına göre.
Arka plan görev	sched, asyncio	Celery, RQ, ARQ, APScheduler	Kalıcı kuyruk gereksinimini ayırın.
Loglama	logging	structlog, OpenTelemetry	Önce standart logging; merkezi sistemde yapılandırılmış log.
Paketleme	zipapp	build, PyInstaller, Nuitka	Kütüphane, wheel veya tek çalıştırılabilir hedefe göre.
Dokümantasyon	pydoc	Sphinx, MkDocs	API ve kullanıcı dokümanı ayrılabilir.

8.1 Bağımlılık seçme kontrolü

- Paketin resmî dokümantasyonu ve sürüm politikası var mı?
- Son sürüm ve Python sürümü uyumluluğu nedir?
- Lisansı projenin dağıtım modeliyle uyumlu mu?
- Paket olmadan standart kütüphane ile makul çözüm mümkün mü?
- Aktif bakım, güvenlik bildirimi ve test kapsamı bulunuyor mu?
- Transitif bağımlılık sayısı ve yerel derleme gereksinimi nedir?
- Windows, Linux ve macOS üzerinde wheel sağlıyor mu?
- Sürüm aralığı sabitlenecek mi, kilit dosyası kullanılacak mı?

9. İşletim Sistemi Farklılıkları

Konu	Windows	Linux / macOS	Taşınabilir çözüm
Python komutu	py -3 veya python	python3 veya python	Kurulum dokümanında iki biçimi gösterin.
venv aktivasyonu	`.\env\Scripts\activate` veya PowerShell Activate.ps1	`source .env/bin/activate`	Aktivasyon zorunlu değildir; ortamın python yolunu doğrudan da çağırabilirsiniz.
Yol ayırıcı	Ters bölü yaygın	Eğik çizgi	pathlib.Path kullanın.
Dosya adı duyarlılığı	Çoğu kurulumda büyük/küçük harfe duyarlı	Genellikle duyarlı	Import ve dosya adlarında tam eşleşme.
Satır sonu	CRLF yaygın	LF yaygın	Git .gitattributes ile standardize edin.
Ortam değişkeni	cmd: set; PowerShell: \$env:NAME	export NAME=value	os.environ veya pydantic-settings.
Kabuk	cmd / PowerShell	bash / zsh	Kabuk bağımlı komutları Python scriptine taşıyın.
Çalıştırılabilir uzantı	exe, bat, ps1	genellikle uzantısız + execute biti	Python entry point kullanın.
Dosya kilitleme	Açık dosyaların silinmesi/yeniden adlandırılması daha kısıtlı olabilir	Unix davranışı farklıdır	Dosyaları context manager ile kapatın.
Sinyaller	POSIX sinyallerinin bir kısmı yoktur	SIGTERM/SIGHUP vb. yaygın	platform ve signal belgelerini kontrol edin.
GUI paketleme	PyInstaller/Nuitka çıktısı Windows'ta üretilmeli	Her hedef sistemde ayrı build	Çapraz paketleme varsaymayın.
Sunucu hedefi	IIS veya Windows Service mümkün	Linux systemd/container çok yaygın	Dağıtım ortamında test edin.

Python'ın venv modülü sanal ortamların oluşturulmasını tanımlar; aktivasyon komutları platforma göre farklıdır. pathlib ise nesne yönelimli dosya sistemi yolları sunar ve platform bağımsız yol yönetimini kolaylaştırır. [7][8]

10. Geliştirme Ortamı ve İlk Kurulum

```
# 1) Proje klasörü
mkdir my_project
cd my_project

# 2) Sanal ortam
python -m venv .venv

# 3) Etkinleştirme
# Windows PowerShell
.venv\Scripts\Activate.ps1
# Linux/macOS
source .venv/bin/activate

# 4) Temel araçlar
python -m pip install --upgrade pip
pip install pytest ruff mypy

# 5) Paket olarak geliştirme
pip install -e .

# 6) Kontrol
pytest
ruff check .
mypy src
```

- Projenin desteklediği Python sürümünü pyproject.toml içindeki requires-python alanında açıkça belirtin.
- Sistem Python ortamına doğrudan uygulama bağımlılığı kurmayın.
- IDE'nin doğru .venv yorumlayıcısını kullandığını doğrulayın.
- Geliştirme ve üretim bağımlılıklarını ayırın; yalnız çalışan kod için gereken paketleri üretime taşıyın.
- Tekrarlanabilir kurulum için bağımlılık sürümlerini proje politikasına uygun sabitleyin ve CI üzerinde temiz ortamda test edin.

11. Veri Katmanı ve Veritabanı Seçimi

Durum	Öneri	Neden
Tek kullanıcı, yerel masaüstü	SQLite	Kurulumuz, tek dosya, standart kütüphane desteği.
Çok kullanıcı web	PostgreSQL	Eşzamanlılık, veri tipleri, güvenilirlik ve ekosistem.
Prototip	SQLite veya geçici bellek	Hızlı başlama; üretim ihtiyaçları ayrıca test edilir.
Analitik sütunlu veri	Parquet + PyArrow/Polars	Sütunlu depolama ve verimli okuma.
Önbellek / kuyruk	Redis	Geçici veri ve görev kuyruğu; ana veritabanı yerine kullanılmamalı.
Belge modeli gerçek ihtiyaç	Belge veritabanı	Şema esnekliği gerçekten gerekiyorsa; ilişkisel modeli erken terk etmeyin.

- Repository arayüzleri iş kurallarını ORM'den ayırabilir.
- Transaction sınırı use case düzeyinde açık olmalıdır.
- Migration olmadan üretim veritabanı şeması elle değiştirilmemelidir.
- Bağlantı bilgileri kod içinde tutulmamalı; ortam değişkenlerinden alınmalıdır.
- SQLAlchemy resmî öğreticisi Core ve ORM kullanımlarını bütünleşik olarak ele alır. [15]

12. Test Stratejisi

Test türü	Amaç	Örnek
Birim testi	Tek fonksiyon/sınıf/iş kuralı; dış bağımlılıklar izole	Net hesaplama kuralı
Entegrasyon testi	DB, dosya sistemi veya API adaptörünün birlikte çalışması	SQLite repository
API testi	HTTP sözleşmesi, durum kodu, şema	POST /books
UI testi	Kritik kullanıcı akışı	Kitap ekleme formu
Uçtan uca	Gerçek sistem zinciri	Kayıt → kitap talebi → teslim
Sözleşme testi	Servisler arası veri formatı	Mesaj şeması
Performans testi	Gecikme, throughput, bellek	10.000 satır rapor
Güvenlik testi	Yetki, girdi, bağımlılık ve gizli bilgi	Rol dışı erişim engeli

```
tests/
├── unit/
├── integration/
├── api/
├── e2e/
├── fixtures/
└── conftest.py
```

- Domain ve application testleri hızlı ve altyapıdan bağımsız olmalıdır.
- Test piramidi; çok sayıda hızlı birim testi, daha az entegrasyon ve sınırlı uçtan uca test önerir.
- Test verisi gerçek kişisel veri içermemelidir.
- Hata düzeltildiğinde aynı hatayı yeniden yakalayan regresyon testi eklenmelidir.

13. Güvenlik, Yapılandırma ve Gizli Bilgiler

- Parola, API anahtarı, veritabanı parolası ve token kod deposuna yazılmaz.
- .env gerçek sırları taşıyabilir ancak .gitignore içinde olmalıdır; .env.example yalnız anahtar adlarını ve güvenli örnekleri içerir.
- Kullanıcı girdileri sınırda doğrulanır; dosya yollarında traversal, SQL'de enjeksiyon, web'de XSS/CSRF riskleri değerlendirilir.
- Kimlik doğrulama ile yetkilendirme ayrı kavramlardır. Kullanıcının kim olduğu ve hangi işlemi yapabildiği ayrı kontrol edilir.
- En az ayrıcalık ilkesi uygulanır; uygulama DB hesabı yalnız gereken izinlere sahip olur.
- Bağımlılıklar düzenli taranır ve güncellenir; kilit dosyası tek başına güvenlik garantisi değildir.
- Kişisel veri asgari tutulur, saklama süresi ve silme süreci belirlenir.
- Üretim hata mesajları kullanıcıya stack trace veya gizli yapılandırma göstermemelidir.

14. Loglama, Hata Yönetimi ve Gözlemlenebilirlik

```
import logging

logger = logging.getLogger(__name__)

def process_book(book_id: int) -> None:
    logger.info("book_processing_started", extra={"book_id": book_id})
    try:
        ...
    except ValueError:
        logger.warning("invalid_book", extra={"book_id": book_id})
        raise
    except Exception:
        logger.exception("book_processing_failed", extra={"book_id": book_id})
        raise
```

- `print()` üretim loglama sistemi değildir. Standart logging modülü esnek olay loglama sağlar. [9]
- Log seviyeleri anlamlı kullanılmalıdır: DEBUG, INFO, WARNING, ERROR, CRITICAL.
- Kişisel veri, parola ve token loglanmamalıdır.
- Korelasyon/istek kimliği, dağıtık sistemlerde bir işlemin izini sürmeye yardımcı olur.
- Beklenen alan hataları özel exception sınıflarıyla; beklenmeyen hatalar merkezi sınırdan ele alınmalıdır.
- Metrik, log ve trace birbirinin yerine geçmez; sistem kritikse birlikte tasarlanır.

15. Paketleme ve Dağıtım

Hedef	Yaklaşım	Araç örnekleri	Not
Python kütüphanesi	wheel/sdist üretilen paket deposuna yükleme	build, twine, PyPI/özel indeks	Semantik sürüm ve genel API önemlidir.
CLI aracı	Python package entry point	pyproject scripts, pipx	Kullanıcı ortamından izole kurulum.
Web API	ASGI/WSGI sunucu + reverse proxy/container	Uvicorn, Gunicorn, Docker	TLS, worker ve sağlık kontrolü.
Django	ASGI/WSGI + statik dosya + DB migration	Gunicorn/Uvicorn, Nginx	DEBUG kapalı, ALLOWED_HOSTS ve secret yönetimi.
Masaüstü	Hedef işletim sisteminde uygulama paketi	PyInstaller, Nuitka, Briefcase	Her OS için ayrı çıktı ve imzalama.
Tek dosyalı script	Kaynak olarak veya zipapp	zipapp	Dış bağımlılık sınırlı olmalı.
Serverless	Fonksiyon paketi veya container	Sağlayıcı araçları	Çalışma süresi ve paket boyutu sınırları.

- Geliştirme makinesinde çalışan uygulama üretime hazır değildir.
- Dağıtım öncesi temiz ortam kurulumu, migration, geri alma ve yedek geri yükleme test edilmelidir.
- FastAPI resmî dağıtım belgeleri, süreç çoğaltma ve worker kavramlarını açıklar. [10]
- Masaüstü uygulama paketleri genellikle hedef işletim sisteminde üretilmelidir.

16. Sürüm Kontrolü ve CI/CD

```

name: quality
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.12", "3.13", "3.14"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${matrix.python-version}
      - run: python -m pip install -e .[dev]
      - run: ruff check .
      - run: mypy src
      - run: pytest --cov

```

- main dalı doğrudan ve kontrolsüz değişikliğe kapatılabilir.
- Her değişiklik küçük, açıklanabilir commitler hâlinde tutulur.
- CI temiz ortamda kurulum, lint, tip kontrolü ve testleri çalıştırır.
- Release süreci etiket, changelog, artefact ve geri alma planı üretmelidir.
- Windows'a özel masaüstü uygulama geliştiriliyorsa CI matrisi Windows runner içermelidir.

17. Mimariye Göre Uygulama ve Fonksiyon Örnekleri

Uygulama	Uygun mimari	Teknolojiler	Temel fonksiyonlar
Dosya düzenleyici	Modüler	pathlib, argparse, logging	Dosyaları kurala göre yeniden adlandırır; dry-run ve geri alma.
Excel okul sonuç analizörü	Pipeline / katmanlı	pandas/openpyxl, pydantic, matplotlib	Dosya doğrulama, temizleme, hesaplama, rapor üretme.
Okul bilgi API'si	Clean + FastAPI	FastAPI, Pydantic, SQLAlchemy, Alembic	Öğrenci/okul kayıtları, rol tabanlı erişim, audit log.
Kitap paylaşım sistemi	Modüler monolit / Django MVT	Django, PostgreSQL	Kayıt, kitap, talep, teslim ve moderasyon.
Proje takip masaüstü uygulaması	Katmanlı + MVVM	PySide6, SQLAlchemy, SQLite	Çevrimdışı kayıt, rapor, şablon ve yedekleme.
Rapor oluşturma servisi	Hexagonal	FastAPI, queue, PDF/Docx araçları	Farklı çıktı adaptörleri ve veri kaynakları.
E-posta bildirim işçisi	Olay güdümlü	Celery/RQ, Redis	Olay tüketip şablonlu bildirim gönderir.
Veri içe aktarma aracı	Pipeline	csv/pandas, Pydantic	Extract → validate → transform → load → report.
Eklentili analiz uygulaması	Plugin	importlib.metadata entry points	Yeni analiz türlerini çekirdeğe dokunmadan yükler.
AI destekli asistan	Katmanlı servis + adaptör	LLM SDK, FastAPI, vector DB ihtiyaca göre	Model sağlayıcısını port/adaptör ile değiştirilebilir tutar.

17.1 Clean/Hexagonal örneği: kitap ekleme use case'i

```
# domain/entities.py
from dataclasses import dataclass

@dataclass(frozen=True)
class Book:
    title: str
    author: str

# application/ports.py
from typing import Protocol

class BookRepository(Protocol):
    def add(self, book: Book) -> int: ...

# application/use_cases.py
class AddBook:
    def __init__(self, repo: BookRepository):
        self.repo = repo

    def execute(self, title: str, author: str) -> int:
        title = title.strip()
        author = author.strip()
        if not title or not author:
            raise ValueError("Başlık ve yazar zorunludur")
        return self.repo.add(Book(title=title, author=author))

# infrastructure/sqlite_repository.py
class SQLiteBookRepository:
    def add(self, book: Book) -> int:
        # sqlite/SQLAlchemy ayrıntısı bu adaptörde kalır
        ...
```

17.2 Pipeline örneği: Excel doğrulama

```
from pathlib import Path
import pandas as pd

def extract(path: Path) -> pd.DataFrame:
    return pd.read_excel(path)

def validate(df: pd.DataFrame) -> pd.DataFrame:
    required = {"okul", "ogrenci", "puan"}
    missing = required - set(df.columns)
    if missing:
        raise ValueError(f"Eksik sütunlar: {sorted(missing)}")
    return df

def transform(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["puan"] = pd.to_numeric(out["puan"], errors="coerce")
    return out.dropna(subset=["puan"])

def run(input_path: Path) -> pd.DataFrame:
    return transform(validate(extract(input_path)))
```

18. Proje Başlangıç Şablonları

18.1 Küçük otomasyon için minimum şablon

```
automation/
├── README.md
├── pyproject.toml
├── src/automation/
│   ├── __init__.py
│   ├── __main__.py
│   └── service.py
└── tests/test_service.py
```

- Önce tek işlev; ikinci bağımsız işlev ortaya çıkmadan katman eklemeyin.
- Dosya sistemi erişimini fonksiyon parametresi veya adaptörle test edilebilir tutun.

18.2 Orta ölçekli iş uygulaması için önerilen şablon

```
business_app/
├── pyproject.toml
├── src/business_app/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   ├── presentation/
│   └── config.py
├── tests/
├── migrations/
├── docs/adr/
└── .github/workflows/
```

- Özellik veya iş alanı büyüdüğünde yatay katmanları modül sınırlarıyla birlikte kullanın.
- ADR (Architecture Decision Record) ile önemli seçimlerin nedeni ve sonuçlarını kaydedin.

18.3 Modüler monolit örneği

```
platform/
├── modules/
│   ├── accounts/
│   │   ├── domain/
│   │   ├── application/
│   │   └── infrastructure/
│   ├── books/
│   └── loans/
├── shared/
├── bootstrap/
└── tests/
```

19. Karar ve Başlangıç Kontrol Listeleri

19.1 Kod yazmadan önce

- Problem ve hedef kullanıcı tek paragrafta açıklanabiliyor mu?
- MVP'nin tek temel akışı tanımlı mı?
- Veri varlıkları ve kişisel veri alanları belli mi?
- Çevrimdışı çalışma gerekiyor mu?
- Tek kullanıcı mı, eşzamanlı çok kullanıcı mı?
- Hedef işletim sistemi ve dağıtım yöntemi belli mi?
- Yedekleme ve veri kaybı senaryosu düşünüldü mü?
- Başarı ölçütü sayısal olarak tanımlandı mı?

19.2 İlk commit içinde

- README.md
- pyproject.toml
- .gitignore
- .env.example
- src/ yapısı
- tests/ ve ilk çalışan test
- Kod biçimlendirme/lint yapılandırması
- Basit CI
- Lisans veya kurum kullanım notu

19.3 Üretime çıkmadan önce

- DEBUG ve geliştirme ayarları kapalı
- Gizli bilgiler dış yapılandırmada
- Migration ve geri alma denendi
- Yedekleme ve geri yükleme denendi
- Yetkilendirme senaryoları test edildi
- Loglarda kişisel veri/sır yok
- Bağımlılık ve güvenlik taraması yapıldı
- Sağlık kontrolü ve gözlemlenebilirlik hazır
- Temiz makinede kurulum testi yapıldı
- Kullanıcı ve yönetici dokümantasyonu güncel

19.4 En sık yapılan hatalar

- Framework klasör yapısını mimari sanmak.
- İş kuralını route, view veya UI event handler içine yazmak.
- Her sınıf için ayrı interface üretip gereksiz soyutlama yapmak.
- Tek kullanıcı SQLite uygulamasına erken mikroservis kurmak.
- Windows'ta geliştirip Linux dosya adı duyarlılığını dikkate almamak.
- requirements/lock dosyasını güncellemeden paket eklemek.
- Gerçek .env dosyasını Git'e göndermek.
- Test olmadan refactor yapmak.

- Veritabanı migration ve yedek geri yükleme planı olmadan üretime çıkmak.
- Kullanılmayan bağımlılıkları projede tutmak.

20. Resmî ve Temel Kaynaklar

Aşağıdaki bağlantılar belgenin teknoloji ve yapılandırma bölümlerinin temel kaynaklarıdır. Mimari desenlerde, desenin özgün yazarının veya çerçevenin resmî açıklaması tercih edilmiştir.

- [1] **Python Documentation** — <https://docs.python.org/3/> — Python dili, standart kütüphane ve öğreticiler.
- [2] **Packaging Python Projects** — <https://packaging.python.org/tutorials/packaging-projects/> — Paket proje yapısı ve gerekli dosyalar.
- [3] **Writing your pyproject.toml** — <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/> — pyproject.toml yapılandırması.
- [4] **FastAPI - Bigger Applications** — <https://fastapi.tiangolo.com/tutorial/bigger-applications/> — Çok dosyalı FastAPI yapısı ve APIRouter.
- [5] **FastAPI - Settings and Environment Variables** — <https://fastapi.tiangolo.com/advanced/settings/> — Ayarları ayrı modülde ve ortam değişkenleriyle yönetme.
- [6] **Django FAQ - MTV architecture** — <https://docs.djangoproject.com/en/6.0/faq/general/> — Django'nun model-template-view adlandırması.
- [7] **Python venv** — <https://docs.python.org/3/library/venv.html> — Sanal ortam ve platforma göre aktivasyon.
- [8] **Python pathlib** — <https://docs.python.org/3/library/pathlib.html> — Platform bağımsız nesne yönelimli dosya yolları.
- [9] **Python logging** — <https://docs.python.org/3/library/logging.html> — Standart loglama sistemi.
- [10] **FastAPI Deployment Concepts** — <https://fastapi.tiangolo.com/deployment/concepts/> — Dağıtım süreçleri ve worker kavramları.
- [11] **Python unittest** — <https://docs.python.org/3/library/unittest.html> — Standart birim test frameworkü.
- [12] **The Clean Architecture - Robert C. Martin** — <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> — Dependency Rule ve içe doğru bağımlılık.
- [13] **Hexagonal Architecture - Alistair Cockburn** — <https://alistair.cockburn.us/hexagonal-architecture> — Ports and Adapters yaklaşımının özgün açıklaması.
- [14] **Qt for Python** — <https://doc.qt.io/qtforpython-6/index.html> — PySide6 ve Qt'nin resmî Python bağları.
- [15] **SQLAlchemy Unified Tutorial** — <https://docs.sqlalchemy.org/tutorial/index.html> — SQLAlchemy Core ve ORM öğreticisi.
- [16] **pytest Documentation** — <https://docs.pytest.org/> — pytest kullanımı ve eklenti ekosistemi.
- [17] **Pydantic Documentation** — <https://docs.pydantic.dev/> — Veri doğrulama ve ayar yönetimi.
- [18] **PyPI** — <https://pypi.org/> — Python paket indeksi.
- [19] **pip Documentation** — <https://pip.pypa.io/> — Paket kurma ve yönetme.
- [20] **Django Documentation** — <https://docs.djangoproject.com/> — Django framework resmî dokümantasyonu.
- [21] **Flask Documentation** — <https://flask.palletsprojects.com/> — Flask resmî dokümantasyonu.

[22] **Ruff Documentation** — <https://docs.astral.sh/ruff/> — Linter ve formatter dokümantasyonu.

[23] **mypy Documentation** — <https://mypy.readthedocs.io/> — Statik tip kontrolü.

[24] **OpenTelemetry Python** — <https://opentelemetry.io/docs/languages/python/> — Trace, metrik ve gözlemlenebilirlik.

Sonuç: Nereden Başlamalı?

21. Problemi ve MVP'yi yazın; teknoloji adı kullanmadan kullanıcı akışını tanımlayın.
22. Uygulama türünü seçin: CLI, API, web, masaüstü, ETL veya kütüphane.
23. En basit sürdürülebilir mimariyle başlayın: küçükte modüler, orta ölçekte katmanlı/modüler monolit, karmaşık iş alanında Clean/Hexagonal/DDD.
24. pyproject.toml, README, src ve tests içeren temiz bir proje iskeleti oluşturun.
25. Sanal ortam kurun; yalnız gerekli paketleri ekleyin.
26. İlk özelliği dikey dilim olarak tamamlayın: giriş → iş kuralı → veri → çıktı → test.
27. CI üzerinde temiz kurulum, lint ve test çalıştırın.
28. Üretimden önce güvenlik, migration, yedekleme, loglama ve geri alma planını doğrulayın.
29. Gerçek ihtiyaç ortaya çıkmadan mikroservis, kuyruk, cache veya karmaşık soyutlama eklemeyin.

Temel ilke: İyi Python uygulaması, en çok paket kullanan veya en çok klasöre sahip uygulama değildir. Değişimi sınırlandıran, test edilebilen, kurulabilen, gözlemlenebilen ve başkası tarafından anlaşılabilen uygulamadır.